# GPU-Accelerated Density Functional Theory Solver for Binary SALR Colloids (SALR3YUCUDA)

Nazar Pasichnyk
*Faculty of Applied Sciences*
*Ukrainian Catholic University*
Lviv, Ukraine
pasichnyk.pn@ucu.edu.ua

Roman Prokhorov
*Faculty of Applied Sciences*
*Ukrainian Catholic University*
Lviv, Ukraine
prokhorov.pn@ucu.edu.ua

Taras Patsahan
*Yukhnovskii Institute for Condensed Matter Physics,*
*NAS of Ukraine*
Lviv, Ukraine
tarpa@icmp.lviv.ua

***Abstract*—We present a computational framework for solving mean-field density functional theory (DFT) equations for spatial density distributions in binary colloidal mixtures with competing short-range attraction and long-range repulsion (SALR) interactions. The SALR pair potential is represented by a three-Yukawa form. The solver implements the Picard iterative procedure for a two-component system under different boundary conditions. Our CPU implementation employs OpenMP parallelization, achieving significant speedup on multi-core processors. A CUDA implementation for GPUs was developed, delivering a multi-fold speedup over the CPU baseline. On the laptop benchmark, the GPU run was $5.15\times$ faster than a 20-thread OpenMP execution, and the cluster run reached $70.85\times$ speedup over a single-thread CPU baseline. We demonstrate the solver capabilities by computing equilibrium density profiles under periodic, two-wall, and four-wall boundary conditions. A Qt-based graphical user interface (GUI) was also built to facilitate parameter configuration, task execution, and result visualization.**

## I. Introduction

Colloidal systems are an important class of soft-matter systems widely used for studying fundamental phenomena such as phase transitions, self-assembly, and the behaviour of complex fluids [**?**]. Particularly intriguing are colloidal systems in which the pair interactions are competing in nature, combining short-range attraction with long-range repulsion (SALR). These interactions, sometimes referred to as "mermaid" potentials because of their attractive head and repulsive tail [**?**]. Systems of particles with SALR interactions can undergo microphase separation, giving rise to rich phase behaviour and mesoscopic structures such as ordered clusters and periodic stripe patterns [**?**]. Figure **??** illustrates a typical SALR potential profile, with the short-range attractive well ("head") followed by a long-range repulsive barrier ("tail").

Despite significant theoretical progress in predicting the behaviour of SALR systems, the experimental realization of the predicted mesophases in three-dimensional real space remains challenging. As Royall notes, "no ordered phase in a system with competing interactions has ever been observed in 3d real space" [**?**]. This gap between theory and experiment motivates the development of efficient computational tools that allow researchers to explore parameter space rapidly and identify conditions under which specific morphologies may emerge.

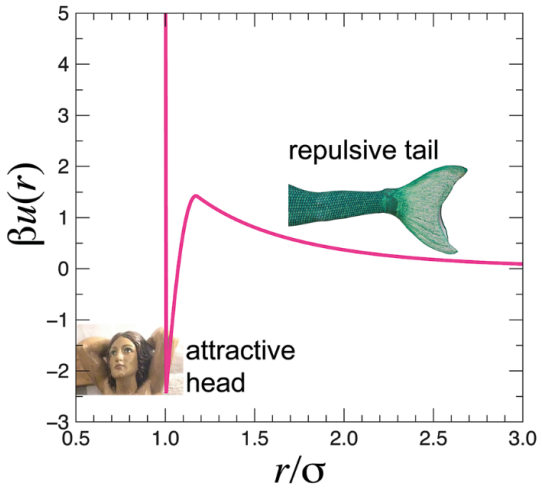


Fig. 1. A "mermaid" (SALR) pair potential as a function of reduced separation $r/\sigma$. The attractive head promotes local condensation while the repulsive tail prevents bulk phase separation, driving the system toward microphase-separated morphologies such as clusters and stripes. Reproduced from [**?**].

Theoretical descriptions of spatial density distributions in SALR systems are commonly based on density functional theory (DFT) within the mean-field approximation [**?**]. This approach leads to a system of nonlinear integral equations that must be solved iteratively. For a two-dimensional grid of $N_x \times N_y$ nodes, the computational cost per iteration scales as $O(N^2)$ due to the convolution required to compute interaction fields, making the problem computationally intensive.

In this work, we present SALR3YUCUDA, a computational solver for mean-field DFT equations describing spatial density distributions in binary mixtures of SALR particles. The project name reflects its key components: SALR (Short-range Attraction Long-range Repulsion), 3Y (three-Yukawa potential), and CUDA (GPU acceleration technology). We implemented an optimized pure C version with OpenMP parallelization for scalable execution on multi-core processors, accompanied by a highly parallelized CUDA framework for GPU-accelerated massive parameter computations. The solvers natively integrate with an SQLite-HDF5 database for session tracking and large spatial tensor storage, and feature multiple boundary

conditions including periodic boundaries (PBC), two parallel walls (W2), and four-wall confinement (W4). Furthermore, a Qt-based graphical interface enables rich visual exploration without cumbersome command-line workflows.

### A. Motivation

Binary colloidal mixtures with SALR interactions are of fundamental interest across soft matter, biophysics, and materials science. The interplay of competing attraction and repulsion gives rise to self-assembled mesophases - clusters, stripes, and lamellar structures - that serve as model systems for understanding protein aggregation, lipid membranes, and block-copolymer microphase separation [**?**], [**?**]. Mapping the conditions under which each morphology appears requires systematic exploration of a high-dimensional parameter space (composition, temperature, interaction strengths and ranges), which is experimentally costly and time-consuming.

Particle-resolved computer simulations such as molecular dynamics (MD) and Monte Carlo (MC) provide the most detailed description of these systems, tracking every particle trajectory and configuration. However, their computational cost scales with the number of particles $N_p$: a single equilibration run for a system large enough to contain several stripe periods may require millions of MC sweeps or nanoseconds of MD trajectory, making large-scale parameter surveys impractical. Furthermore, finite-size effects are pronounced when the system box is not much larger than the characteristic pattern wavelength.

Density functional theory offers a complementary approach that avoids these limitations. Rather than evolving individual particle positions, DFT works directly with continuous density fields $\rho_i(\mathbf{r})$, reducing an $N_p$-body statistical mechanics problem to a deterministic nonlinear integral equation solved on an $N_x \times N_y$ grid. Key advantages include:

- **Direct equilibrium access:** The iterative solver finds the equilibrium density profile directly without the need for thermalization or ensemble averaging inherent to stochastic simulations.
- **Clean boundary conditions:** Walled geometries are imposed analytically as Dirichlet constraints on $\rho_i$, avoiding surface-energy artifacts from finite particle layers.
- **Parameter scalability:** Changing temperature, density, or interaction coefficients requires only reinitializing the solver, enabling rapid sweeps across the phase diagram.

These properties make mean-field DFT the method of choice for initial reconnaissance of SALR phase behavior, providing guidance for more expensive particle-level studies and for comparison with theory. The present work implements and accelerates this approach, with GPU parallelization targeting the dominant $O(N^2)$ convolution kernel to rapidly expand the accessible parameter space.

## II. Theoretical Background

### A. SALR Interactions

Short-range attraction and long-range repulsion (SALR) interactions arise naturally in colloidal systems where particles experience depletion attraction at short distances (due to polymer or small particle exclusion) and electrostatic repulsion at longer distances [**?**]. The competition between these interactions leads to complex energy landscapes and the possibility of microphase separation, where the system forms periodic structures such as clusters, stripes, or lamellae rather than undergoing bulk phase separation. Of particular relevance to the present work, Kravtsiv et al. showed that soft-core fluids with competing interactions confined near hard walls exhibit enhanced cluster formation relative to the bulk, with distinct density layering driven purely by confinement geometry [**?**]. This motivates our inclusion of hard-wall boundary conditions as a physically meaningful test case for the solver.

### B. Three-Yukawa Potential

We model the pair interaction between particles of species $i$ and $j$ using a three-Yukawa form:

$$U_{ij}(r) = \sum_{m=1}^{3} A_{ij}^{(m)} \frac{\exp(-\alpha_{ij}^{(m)} r)}{r}, \quad 0 < r \le r_c \tag{1}$$

where $A_{ij}^{(m)}$ are amplitude coefficients, $\alpha_{ij}^{(m)}$ are inverse screening lengths, and $r_c$ is a cutoff radius beyond which $U_{ij}(r) = 0$. The three Yukawa terms allow flexible representation of the SALR shape: positive amplitudes at short inverse lengths create repulsion, while negative amplitudes create attraction.

For a binary mixture (species 1 and 2), we require three independent potentials: $U_{11}(r)$ for species 1–1 interactions, $U_{22}(r)$ for species 2–2 interactions, and $U_{12}(r) = U_{21}(r)$ for cross-species interactions.

### C. Mean-Field Density Functional Theory

Within the mean-field approximation, the grand thermodynamic potential functional for the binary mixture leads to Euler-Lagrange equations that determine the equilibrium density profiles $\rho_1(\mathbf{r})$ and $\rho_2(\mathbf{r})$:

$$\rho_1(\mathbf{r}) = \rho_{1,b} \exp[-\beta\,(\Phi_{11}(\mathbf{r}) + \Phi_{12}(\mathbf{r}) - \Phi_{11,b} - \Phi_{12,b})] \tag{2}$$

$$\rho_2(\mathbf{r}) = \rho_{2,b} \exp[-\beta\,(\Phi_{21}(\mathbf{r}) + \Phi_{22}(\mathbf{r}) - \Phi_{21,b} - \Phi_{22,b})] \tag{3}$$

where $\beta = 1/(k_B T)$ is the inverse temperature (with $k_B$ the Boltzmann constant), $\rho_{i,b}$ is the bulk (average) density of species $i$, and $\Phi_{ij}(\mathbf{r})$ are interaction field contributions defined as convolutions:

$$\Phi_{ij}(\mathbf{r}) = \int d\mathbf{r}'\, \rho_j(\mathbf{r}')\, U_{ij}(|\mathbf{r} - \mathbf{r}'|) \tag{4}$$

The bulk field values $\Phi_{ij,b}$ represent the interaction fields in a uniform system with densities $\rho_{1,b}$ and $\rho_{2,b}$, ensuring that the trivial homogeneous solution satisfies the equations.

### D. Picard Iteration

Equations (**??**)–(**??**) are solved iteratively using the Picard method with mixing:

$$\rho_i^{(t+1)}(\mathbf{r}) = \xi_i K_i[\rho^{(t)}](\mathbf{r}) + (1-\xi_i)\rho_i^{(t)}(\mathbf{r}) \tag{5}$$

where $K_i[\rho]$ denotes the right-hand side of the Euler-Lagrange equation and $\xi_i \in (0,1]$ is a mixing parameter that controls convergence stability. Iterations continue until the root-mean-square change falls below a tolerance $\varepsilon$:

$$\left\|\rho_i^{(t+1)} - \rho_i^{(t)}\right\| = \sqrt{\frac{1}{N_x N_y}\sum_k \left[{\rho_i^{(t+1)}}_k - {\rho_i^{(t)}}_k\right]^2} < \varepsilon \tag{6}$$

### E. Boundary Conditions

The solver supports three boundary modes:

- **PBC (Periodic Boundary Conditions):** The domain wraps around in both $x$ and $y$ directions, simulating a bulk-like infinite system.
- **W2 (Two Walls):** Hard walls at $x = 0$ and $x = L_x$ with periodic boundaries in $y$. The density is forced to zero at wall positions: $\rho_i(x = 0) = \rho_i(x = L_x) = 0$.
- **W4 (Four Walls):** Hard walls on all boundaries, confining particles to a rectangular box.

## III. Implementation

### A. Discretization

The two-dimensional domain $[0, L_x] \times [0, L_y]$ is discretized on a uniform grid with $N_x \times N_y$ nodes and spacing $\Delta x$, $\Delta y$. The continuous convolution integral (**??**) becomes a discrete sum:

$$\Phi_{ij}(x_i, y_j) = \Delta x \Delta y \sum_{m,n} \rho_j(x'_m, y'_n)\, U_{ij}(|\mathbf{r}_{ij} - \mathbf{r}'_{mn}|) \tag{7}$$

For efficiency, the potential $U_{ij}(r)$ is precomputed and stored in lookup tables indexed by displacement $(d_{ix}, d_{iy})$ between grid cells.

### B. Algorithm Overview

The solver proceeds through the following steps:

1) **Initialization:** Load parameters; build potential tables $U_{11}$, $U_{12}$, $U_{22}$; compute bulk fields $\Phi_{ij,b}$; set initial density profiles with small perturbations.
2) **Iteration loop:**
   a) Compute interaction fields $\Phi_{11}$, $\Phi_{12}$, $\Phi_{21}$, $\Phi_{22}$ via discrete convolution.
   b) Evaluate $K_1$ and $K_2$ using equations (**??**)–(**??**).
   c) Apply mass conservation renormalization.
   d) Perform Picard mixing (**??**).
   e) Apply boundary conditions (zero density at walls).
   f) Apply anti-aliasing smoothing to suppress numerical artifacts.
   g) Check convergence criterion (**??**).
3) **Output:** Save converged density profiles.

### C. CPU Parallelization

The most computationally expensive operation is the $O(N^2)$ convolution for computing $\Phi_{ij}$. We employ OpenMP to parallelize:

- Potential table construction (loop over grid offsets)
- Convolution computation (nested loops over source and field cells)
- $K$-operator evaluation and Picard mixing

On a multi-core processor (Intel Core i7-13700H, 20 threads), this OpenMP parallelization achieves a multi-fold speedup compared to single-threaded execution on the largest grid sizes tested.

### D. CUDA Implementation

GPU acceleration explicitly targets the critical convolution operation by offloading both potential generation and density mixing to the CUDA kernel pipeline. We employ coalesced global memory access and constant memory for potential parameters (such as the Yukawa amplitudes and inverse screening lengths) to maximize global bandwidth throughput. Custom 2D thread blocks iteratively reduce memory fetches during the spatial convolution loops.

### E. GUI and Session Database

The project integrates a Qt 6-based graphical interface for end-to-end simulation lifecycle management. Parameters such as temperature, grid shape, and boundary constraints are configured through the GUI, while the backend stores sessions and density snapshots in SQLite and HDF5 for later inspection and reuse.

## IV. Results on our Machines

### A. Simulation Parameters

We demonstrate the solver using the parameters listed in Table **??**. These values match the current default configuration used by the code and the generated benchmark figures.

TABLE I
Simulation Parameters

| Parameter | Value |
|---|---|
| System size $L_x \times L_y$ | $16.0 \times 16.0$ |
| Grid resolution $N_x \times N_y$ | $160 \times 160$ |
| Grid spacing $\Delta x = \Delta y$ | 0.1 |
| Temperature $T$ | 8.0 |
| Bulk density $\rho_{1,b}$ | 0.2 |
| Bulk density $\rho_{2,b}$ | 0.2 |
| Cutoff radius $r_c$ | 8.0 |
| Mixing coefficients $\xi_1 = \xi_2$ | 0.02 |
| Maximum iterations | 50000 |
| Tolerance $\varepsilon$ | $10^{-8}$ |
| Boundary mode | PBC |
| Init mode | sinusoids |

The three-Yukawa coefficients for species 1–1 interaction are $A_{11}^{(1)} = 150.66$, $A_{11}^{(2)} = -122.61$, $A_{11}^{(3)} = 27.12$ with corresponding $\alpha_{11}^{(1)} = 1.92$, $\alpha_{11}^{(2)} = 1.26$, $\alpha_{11}^{(3)} = 0.76$. These parameters produce a potential with attractive head and repulsive tail characteristic of SALR systems.

### B. Periodic Boundary Conditions (PBC)

*1) Random starting distribution:* For PBC runs, the final pattern depends strongly on the initial distribution. A random start tends to produce large chaotic structures that break translational symmetry at multiple length scales. This behavior is consistent with the sensitive SALR free energy landscape, where multiple local minima exist.

**Placeholder**
Provide PBC random-start snapshot (combined 2D and 3D view).
File name: src/pbc-random.png

Fig. 2. PBC with random starting distribution. Placeholder for a combined 2D and 3D snapshot.

*2) Sinusoidal starting distribution:* A sinusoidal start more often converges to diagonal stripe patterns. The regular initial modulation seeds a single dominant wavelength, producing cleaner microphase ordering and fewer competing domains.

**Placeholder**
Provide PBC sinusoidal-start snapshot (combined 2D and 3D view).
File name: src/pbc-sinusoidal.png

Fig. 3. PBC with sinusoidal starting distribution. Placeholder for a combined 2D and 3D snapshot.

*3) Representative snapshot (init mode not recorded):* Figure **??** shows a representative PBC snapshot from the current results set. The image includes both the 3D scatter plot and the 2D joint heatmap. The init mode was not recorded in the snapshot, so it is shown here as an example of a converged PBC morphology only. These representative snapshots use a different parameter set ($T = 2.90$, $\rho_1 = 0.40$, $\rho_2 = 0.20$) and are included for qualitative illustration only.

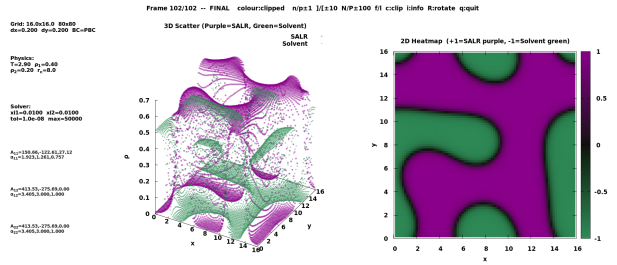


Fig. 4. Representative PBC snapshot (combined 2D and 3D view). Parameters shown in the image: $T = 2.90$, $\rho_1 = 0.40$, $\rho_2 = 0.20$, $r_c = 8.0$, grid $80 \times 80$ with $\Delta x = \Delta y = 0.2$. Init mode not recorded.

### C. Two-Wall Confinement (W2)

*1) Random starting distribution:* Under W2 confinement, the walls suppress density at $x = 0$ and $x = L_x$, which drives the system toward wall-parallel modulation. Random starting distributions tend to produce uneven wall layers before relaxing into a more regular pattern.

**Placeholder**
Provide W2 random-start snapshot (combined 2D and 3D view).
File name: src/w2-random.png

Fig. 5. W2 with random starting distribution. Placeholder for a combined 2D and 3D snapshot.

*2) Sinusoidal starting distribution:* With sinusoidal starting distributions, the solver tends to keep stripe orientation parallel to the walls, and the boundary layers stabilize faster due to the imposed spatial modulation.

**Placeholder**
Provide W2 sinusoidal-start snapshot (combined 2D and 3D view).
File name: src/w2-sinusoidal.png

Fig. 6. W2 with sinusoidal starting distribution. Placeholder for a combined 2D and 3D snapshot.

*3) Representative snapshot (init mode not recorded):* Figure **??** shows a representative W2 snapshot from the current results set. The init mode was not recorded in the snapshot, so it is shown here as an example of the boundary-driven morphology only. The parameter set matches the PBC representative snapshot.

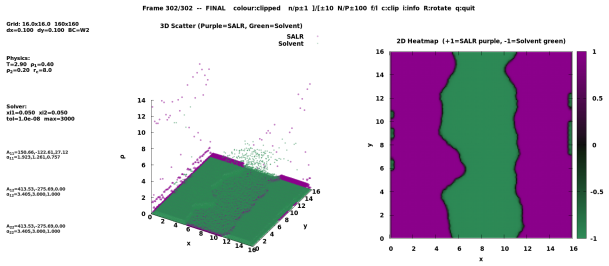


Fig. 7. Representative W2 snapshot (combined 2D and 3D view). Parameters shown in the image: $T = 2.90$, $\rho_1 = 0.40$, $\rho_2 = 0.20$, $r_c = 8.0$, grid $160 \times 160$ with $\Delta x = \Delta y = 0.1$. Init mode not recorded.

### D. Four-Wall Confinement (W4)

*1) Random starting distribution:* In the four-wall case, the random start often creates a dense core surrounded by a depleted boundary layer. Corner depletion becomes more pronounced as the field relaxes toward equilibrium.

**Placeholder**
Provide W4 random-start snapshot (combined 2D and 3D view).
File name: src/w4-random.png

Fig. 8. W4 with random starting distribution. Placeholder for a combined 2D and 3D snapshot.

*2) Sinusoidal starting distribution:* For sinusoidal starts, the confined system tends to form a smoother central profile with stronger boundary depletion. The walls fix the pattern phase and reduce the number of competing domains.

**Placeholder**
Provide W4 sinusoidal-start snapshot (combined 2D and 3D view).
File name: src/w4-sinusoidal.png

Fig. 9. W4 with sinusoidal starting distribution. Placeholder for a combined 2D and 3D snapshot.

*3) Representative snapshot (init mode not recorded):* Figure **??** shows a representative W4 snapshot from the current results set. The init mode was not recorded in the snapshot, so it is shown here as an example of a confined morphology only. The parameter set matches the PBC representative snapshot.

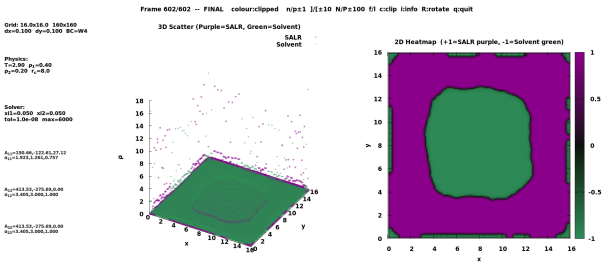


Fig. 10. Representative W4 snapshot (combined 2D and 3D view). Parameters shown in the image: $T = 2.90$, $\rho_1 = 0.40$, $\rho_2 = 0.20$, $r_c = 8.0$, grid $160 \times 160$ with $\Delta x = \Delta y = 0.1$. Init mode not recorded.

## *E. Performance on our Machines*

*1) Measured metrics:* We focus on two primary metrics: *execution time* and *speedup*.

**Execution time** is measured as the total wall-clock time from the start of the iteration loop to convergence (or the maximum iteration limit). It is the most direct indicator of solver efficiency and directly determines how quickly a researcher can sweep across the parameter space. Disk I/O is excluded to isolate the compute-only cost.

**Speedup** is defined as:

$$S = \frac{t_{\text{ref}}}{t_{\text{target}}} \tag{8}$$

where $t_{\text{ref}}$ is the reference execution time (typically single-thread CPU or a fixed baseline) and $t_{\text{target}}$ is the execution time of the configuration under evaluation. We report two kinds of speedup: CUDA over CPU (to quantify GPU benefit) and multi-thread CPU over single-thread CPU (to quantify OpenMP scaling efficiency). These two metrics together reveal both absolute performance gains and scaling behavior as compute resources increase.

*2) Benchmark parameters and hardware:* The performance benchmarks use a dedicated benchmark configuration with disk I/O disabled. The same physical parameters as the default solver run are used. Grid sizes and thread counts follow the measured dataset from the laptop benchmark logs:

- Grid sizes: $N \in \{16, 32, 64, 128, 160, 256, 320\}$ (square grids, $N \times N$ nodes).
- CPU threads: $\{1, 2, 4, 8, 12, 16, 20\}$ and CUDA on the GPU.
- Max iterations: 1000, tolerance $10^{-8}$, boundary mode PBC, init mode sinusoids.

Laptop hardware used for the benchmark run is summarized in Table **??**. The CPU is an Intel Core i7-13700H featuring a hybrid architecture with 6 performance cores (P-cores, optimized for single-thread throughput at up to 5.0 GHz) and 8 efficiency cores (E-cores, optimized for parallelism at lower power), totaling 20 logical threads under HyperThreading. The three-level cache hierarchy (L1/L2 per-core, shared L3) determines data reuse efficiency for the convolution loops. The DDR5-4800 memory provides high bandwidth for large grid transfers. The GPU is an NVIDIA RTX 4060 Laptop based on the Ada Lovelace architecture. Its 3072 CUDA cores execute floating-point operations in massively parallel warps. The 8 GB GDDR6 VRAM with 128-bit bus delivers 272 GB/s bandwidth, which is the main performance bottleneck for the memory-bound convolution kernel. The GPU L2 cache (24 MiB) helps reuse potential table data across thread blocks. Peak FP32 throughput is 14.55 TFLOPS, roughly $35\times$ the CPU peak.

TABLE II
LAPTOP HARDWARE USED FOR BENCHMARKS

| Parameter | Value |
|---|---|
| CPU model | Intel Core i7-13700H |
| CPU cores / threads | 14 cores (6P + 8E) / 20 threads |
| CPU max frequency | 5.0 GHz |
| L1d / L1i cache | 544 KiB / 704 KiB (14 instances) |
| L2 cache | 11.5 MiB (8 instances) |
| L3 cache | 24 MiB (shared) |
| RAM | 16 GB DDR5-4800 (2×8 GB SODIMM) |
| GPU model | NVIDIA GeForce RTX 4060 Laptop GPU |
| GPU architecture | Ada Lovelace |
| CUDA cores | 3072 |
| GPU boost clock | 2370 MHz |
| VRAM capacity | 8 GB GDDR6 |
| VRAM bus width | 128-bit |
| Memory bandwidth | 272 GB/s |
| FP32 throughput | 14.55 TFLOPS |
| GPU cache (L2) | 24 MiB |

*3) Methodology:* Each $(N, \text{threads})$ configuration was executed three times, and the median wall time was recorded to CSV by the benchmark script. Taking the median over three runs reduces the effect of transient OS scheduling noise and cache warm-up variation. Disk I/O was disabled via a large save interval so that timing captures only compute. CUDA and CPU runs use identical grids, physical parameters, and convergence criteria to ensure fair comparison.

Speedup values are computed post-hoc from the CSV data. For CUDA vs. CPU comparisons, the single-thread CPU time at each grid size serves as the baseline. For OpenMP scaling, the 1-thread CPU time at each grid size is the reference. Grid sizes for which only CUDA data exist (no CPU measurement at that $N$) are included in the CUDA curves but excluded from speedup ratios that require a CPU counterpart.

*4) Benchmark results:* Figure **??** reports execution time versus grid size for all CPU thread counts and CUDA. CUDA is slower for the smallest grid ($N = 16$) due to launch overheads, but becomes faster from $N = 32$ onward. At $N = 160$, CUDA is about $5.0\times$ faster than the 20-thread CPU run. CPU timing for $N = 256$ and $N = 320$ is not recorded in the benchmark logs, so those points appear only for CUDA.

Figure **??** shows CUDA speedup versus each CPU baseline and the CPU parallel scaling relative to a single thread. CPU speedup saturates near 7 to $8\times$ at the largest grid sizes, indicating memory bandwidth and synchronization limits.

The convergence check in Figure **??** confirms that CPU and CUDA reach the same tolerance and iteration count for the default physical parameters.

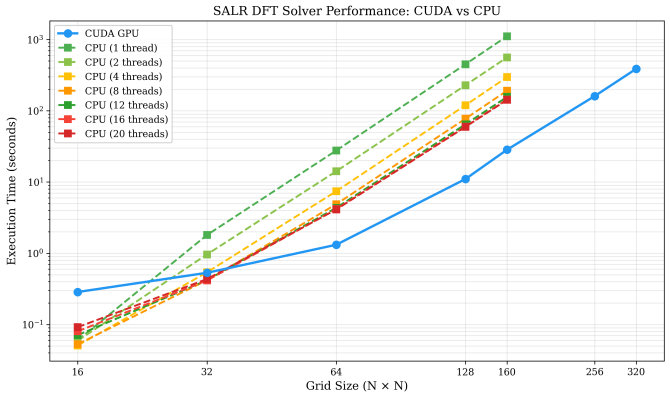


Fig. 11. Laptop benchmark runtime for CUDA and CPU across grid sizes.

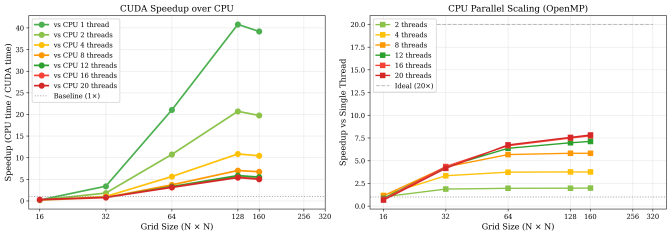


Fig. 12. Laptop benchmark speedups: CUDA over CPU (left) and CPU OpenMP scaling (right).

## V. Results on Cluster

### A. Benchmark setup

Cluster benchmarks use the same benchmark configuration as the laptop run, with I/O disabled and 1000 iterations per configuration. The dataset contains three machine classes:

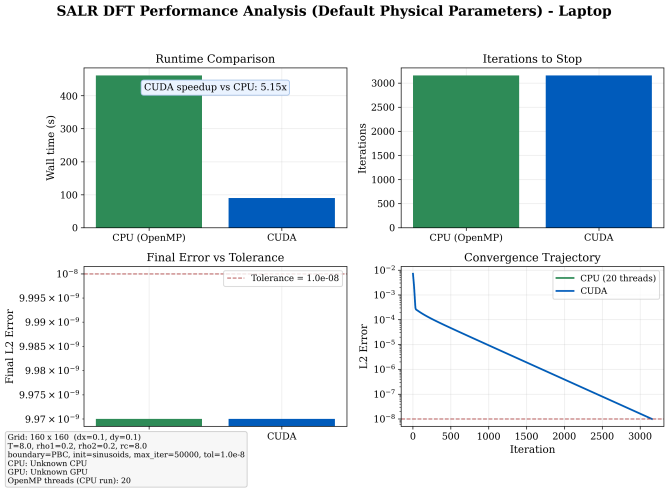


Fig. 13. Laptop convergence comparison for default physical parameters. CPU and CUDA converge to the same error threshold with identical iteration counts.

- AMD EPYC 9754 (CPU only, no NVIDIA GPU detected).
- AMD Threadripper PRO 5975WX (CPU only, no NVIDIA GPU detected).
- AMD Threadripper PRO 5975WX with NVIDIA GeForce RTX 5090.

All benchmark CSV files record the same grid sizes ($N = 16$ to 160). EPYC thread counts include $\{1, 2, 4, 8, 16, 32, 64\}$, while the Threadripper datasets include $\{1, 2, 4, 8, 16\}$. The EPYC dataset does not include a 1-thread measurement at $N = 160$.

### B. Hardware summary

Cluster hardware details captured by the benchmark logs are summarized in Table **??**. The AMD EPYC 9754 is a server-grade processor in a dual-socket configuration ($2\times128$ cores, 256 cores total), designed for high-core-count parallel workloads. Its large core count allows the OpenMP convolution to distribute across many threads with minimal per-thread overhead. The AMD Threadripper PRO 5975WX is a workstation processor with 32 cores (64 threads), well suited for compute-intensive desktop workloads. The RTX 5090 is NVIDIA's flagship Blackwell-generation GPU with 32 GB of VRAM, enabling very large grid runs and significantly higher theoretical throughput than the RTX 4060 Laptop. Additional hardware properties such as cache sizes and memory bandwidth for the cluster nodes were not recorded by the benchmark logs.

### C. Performance across cluster nodes

Figure **??** compares the best CPU time per grid size across the laptop and cluster nodes on a log scale. The EPYC 9754 achieves the lowest CPU runtimes due to its high core count, with 64-thread scaling yielding the best absolute times. The Threadripper PRO 5975WX shows intermediate performance with up to 16 threads.

TABLE III
CLUSTER HARDWARE USED FOR BENCHMARKS. NODES WERE ALLOCATED EXCLUSIVELY TO ENSURE CLEAN TIMING RESULTS.

| Node (queue) | CPU model | CPU cores | Max threads used | GPU | VRAM |
|---|---|---|---|---|---|
| node-003 () | AMD EPYC 9754 | 256 (2×128) | 64 | None | N/A |
| gnode-003 () | AMD Threadripper PRO 5975WX | 32 | 16 | None | N/A |
| gnode-003 () | AMD Threadripper PRO 5975WX | 32 | 1 (host) | NVIDIA GeForce RTX 5090 | 32607 MiB |

Figure **??** summarizes OpenMP thread-scaling speedup at the largest available grid size for each machine. The EPYC node scales to 64 threads; the Threadripper node reaches 16. Both plateau well below the linear ideal, consistent with the memory-bound nature of the $O(N^2)$ convolution.

Figures **??** and **??** compare GPU versus CPU runtimes and show speedup of each machine relative to the single-thread laptop baseline. The RTX 5090 on the cluster GPU node delivers the best GPU performance, reflecting both the larger VRAM and higher CUDA throughput compared to the laptop RTX 4060.

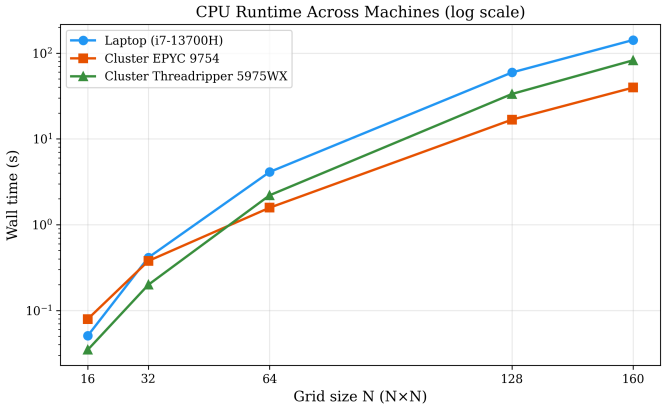


Fig. 14. Best CPU runtime across laptop and cluster machines (log scale).

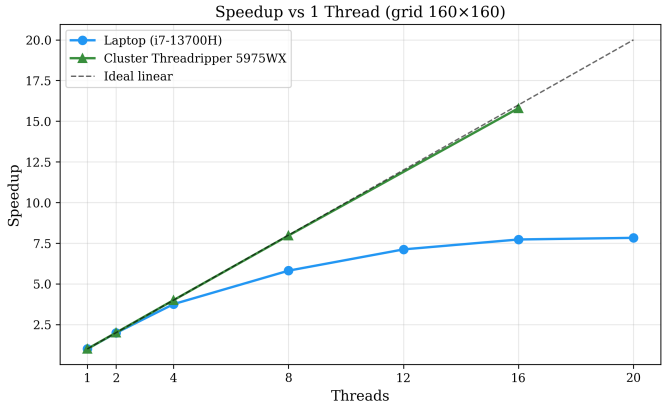


Fig. 15. OpenMP speedup versus single-thread baseline for the largest available grid size.

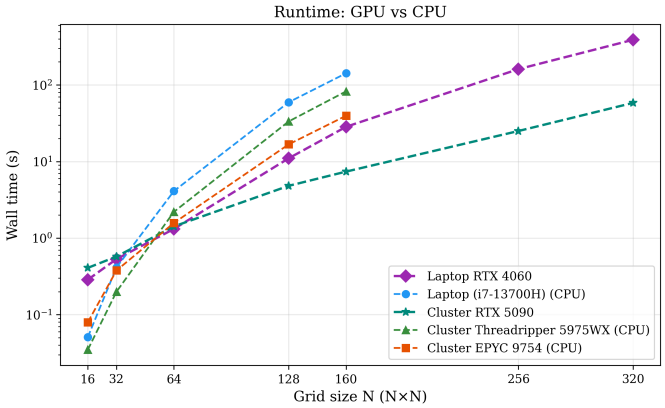


Fig. 16. GPU and CPU runtime comparison across machines. CPU-only nodes are shown without GPU curves.

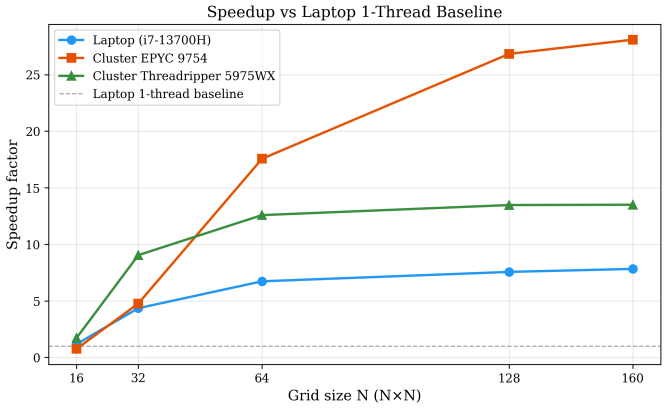


Fig. 17. Speedup of each machine relative to the laptop single-thread baseline.

### *D. Convergence check on the cluster GPU*

Figure **??** summarizes the convergence comparison for the RTX 5090 run with a single-thread CPU baseline. The iteration counts and final error match, confirming numerical equivalence between CPU and CUDA implementations.

## VI. DISCUSSION

The computed density distributions demonstrate that our solver captures the essential physics of SALR systems: microphase separation driven by competing interactions, and the influence of confinement on pattern morphology. The results also confirm that initial conditions can steer the solver toward distinct local minima. Random starts frequently generate

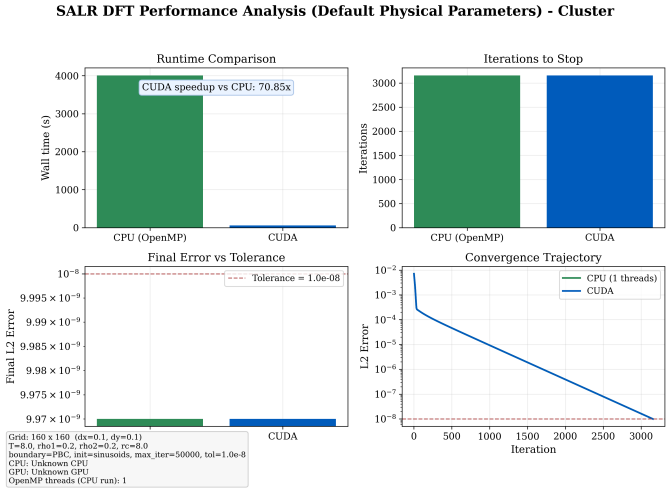


Fig. 18. Cluster convergence comparison for default physical parameters (Threadripper PRO 5975WX with RTX 5090).

chaotic structures, while sinusoidal starts stabilize diagonal stripes more reliably.

At low temperatures, the density field can develop point-like spikes that appear gradually during iteration. We do not yet know whether these are physical microstructures or numerical artifacts, but their presence suggests that additional verification is required for low $T$ runs.

In the direct integration algorithm, we observed accumulation of numerical error that drives the solution toward near-zero uniform density or produces inf and NaN values. Adding normalization and Laplace smoothing mitigates most failures, but the result remains sensitive to the smoothing coefficient, so it must be tuned carefully.

Performance data highlight the benefits of GPU acceleration and the limits of CPU scaling. On the laptop, CUDA is slower only for the smallest grid ($N = 16$), then surpasses all CPU thread counts for larger grids and reaches about a $5.0\times$ speedup at $N = 160$. On the cluster RTX 5090 run, the CUDA solver achieves a $70.85\times$ speedup over a single-thread CPU baseline for the same physical parameters. CPU speedup saturates near 7 to $8\times$ at the largest grids, indicating bandwidth and synchronization bottlenecks.

## VII. Conclusion

We have presented SALR3YUCUDA, a computational solver for mean-field DFT equations describing binary SALR colloidal mixtures. The CPU implementation with OpenMP parallelization and the CUDA accelerator both compute equilibrium density distributions under periodic and confined boundary conditions, capturing the stripe-like morphologies expected for competing interactions.

The benchmark set shows that GPU acceleration provides substantial gains. For the laptop dataset, CUDA reaches convergence about $5.15\times$ faster than the 20-thread CPU baseline for the default physical parameters. On the cluster RTX 5090 run, CUDA delivers a $70.85\times$ speedup over a single-thread CPU baseline. The integrated Qt GUI serves as a practical pipeline for iterative testing and on-the-fly visualization of distributions.

Future work will expand horizontally along several vectors including:

- Extension to true three-dimensional domains.
- Multi-GPU scaling via deep domain decomposition.
- Automated phase-diagram extraction algorithms mapping extensive structural regions seamlessly across the interactive GUI.

## Acknowledgment

This work was performed as part of the Architecture of Computer Systems course at Ukrainian Catholic University. We thank the course instructors for guidance on parallelization strategies.

Computer time for the reported simulations was provided by the Interdisciplinary Center for Computer Simulations (Yukhnovskii Institute for Condensed Matter Physics of the National Academy of Sciences of Ukraine).

## References

[1] C. P. Royall, “Hunting mermaids in real space: known knowns, known unknowns and unknown unknowns,” *Soft Matter*, vol. 14, no. 20, pp. 4020–4028, 2018.

[2] Y. Liu and Y. Xi, “Colloidal systems with a short-range attraction and long-range repulsion: Phase diagrams, structures, and dynamics,” *Current Opinion in Colloid & Interface Science*, vol. 39, pp. 123–136, 2019.

[3] J.-P. Hansen and I. R. McDonald, *Theory of Simple Liquids: With Applications to Soft Matter*, 4th ed. Academic Press, 2013.

[4] Chacko, B., Chalmers, C. & Archer, A. Two-dimensional colloidal fluids exhibiting pattern formation. *The Journal Of Chemical Physics*. **143** (2015)

[5] Pini, D. & Parola, A. Pattern formation and self-assembly driven by competing interactions. *Soft Matter*. **13**, 9259-9272 (2017)

[6] Kravtsiv, I., Patsahan, T., Holovko, M. & Di Caprio, D. Soft core fluid with competing interactions at a hard wall. *Journal Of Molecular Liquids*. **362** pp. 119652 (2022)

[7] NVIDIA Corporation, “CUDA C++ Programming Guide,” v12.4, 2024. [Online]. Available: https://docs.nvidia.com/cuda/cuda-programming-guide/. [Accessed: 7-May-2026].

[8] Roth, R. Introduction to density functional theory of classical systems: Theory and applications. *Lecture Notes, November*. (2006)